

webui\webui\_server.py

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  @file      webui_server.py
5  @brief     Serveur web pour interface utilisateur de configuration.
6  @details   Interface web Flask monopage pour lecture/écriture de la configuration
7             sensors.json et affichage de l'état last_state.json. Fournit API REST
8             complète pour gestion configuration canaux et monitoring système.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 # Import du module système pour gestion des chemins
18 import os
19 # Import du module JSON pour sérialisation des données
20 import json
21 # Import pour gestion des horodatages
22 from datetime import datetime
23 # Imports Flask pour serveur web et API REST
24 from flask import Flask, request, jsonify, send_from_directory
25
26
27 # -----
28 # Configuration des chemins et constantes
29 # -----
30
31 # Répertoire de base adapté pour Windows - chemin relatif au script
32 BASE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
33 # Chemin vers le fichier de configuration des capteurs
34 CONFIG_PATH = os.path.join(BASE_DIR, "config_module", "sensors.json")
35 # Chemin vers le fichier d'état du système
36 STATE_PATH = os.path.join(BASE_DIR, "state", "last_state.json")
37 # Répertoire des fichiers statiques de l'interface web
38 WEBUI_DIR = os.path.dirname(__file__)
39
40 # Création de l'instance Flask avec configuration des fichiers statiques
41 app = Flask(__name__, static_folder=WEBUI_DIR, static_url_path='/static')
42
43
44 # -----
45 # Routes de l'interface web
46 # -----
47
48 @app.route("/", methods=["GET"])
49 def serve_index():
50     """
51     @brief   Sert la page principale de l'interface web.
52     @details Route racine qui retourne le fichier index.html depuis
53             le répertoire webui pour affichage de l'interface utilisateur.
54
55     @return  Fichier HTML de l'interface principale.
56     """
57     # Retour du fichier index.html depuis le répertoire webui
58     return send_from_directory(WEBUI_DIR, "index.html")
59
60
61 # -----
62 # API REST pour gestion de la configuration
63 # -----
64
65 @app.route("/api/config", methods=["GET"])
66 def get_config():
67     """
68     @brief   Récupère la configuration actuelle des capteurs.
69     @details Lecture du fichier sensors.json et retour des données
70             de configuration en format JSON. Gère les erreurs de
71             lecture et affiche des informations de débogage.
72
73     @return  Configuration JSON ou message d'erreur avec code HTTP.
74     """
75     # Bloc de traitement avec gestion d'exceptions
76     try:
77         # Log de l'opération de lecture avec chemin complet
78         print(f"[WEBUI] Lecture configuration depuis {CONFIG_PATH}")
79         # Ouverture du fichier de configuration en UTF-8
80         with open(CONFIG_PATH, "r", encoding="utf-8") as f:
81             # Chargement des données JSON depuis le fichier
82             data = json.load(f)
83             # Log du nombre de canaux chargés pour validation
84             print(f"[WEBUI] Configuration chargée: {len(data.get('channels', []))} canaux")
```

```

85         # Retour des données avec code HTTP 200 (succès)
86         return jsonify(data), 200
87     except Exception as e:
88         # Log de l'erreur rencontrée lors de la lecture
89         print(f"[WEBUI] Erreur lecture config: {e}")
90         # Retour d'erreur avec code HTTP 500 (erreur serveur)
91         return jsonify({"error": str(e)}), 500
92
93 @app.route("/api/config", methods=["PUT"])
94 def put_config():
95     """
96     @brief Sauvergarde une nouvelle configuration des capteurs.
97     @details Reçoit des données JSON, valide le contenu et sauvegarde
98             dans sensors.json avec sauvegarde atomique via fichier temporaire.
99             Affiche un résumé des changements principaux.
100
101     @return Statut de sauvegarde ou message d'erreur avec code HTTP.
102     """
103     # Bloc de traitement avec gestion d'exceptions
104     try:
105         # Récupération des données JSON depuis la requête HTTP
106         payload = request.get_json(force=True, silent=False)
107         # Log du nombre de canaux à sauvegarder
108         print(f"[WEBUI] Sauvegarde configuration: {len(payload.get('channels', []))} canaux")
109
110         # Section d'affichage des changements principaux de configuration
111         if 'mac_address' in payload:
112             # Log de la nouvelle adresse MAC si présente
113             print(f"[WEBUI] MAC: {payload['mac_address']}")
114         if 'auto_sequence' in payload:
115             # Log du mode séquence automatique si présent
116             print(f"[WEBUI] Auto séquence: {payload['auto_sequence']}")
117
118         # Filtrage et affichage des canaux activés seulement
119         enabled_channels = [ch for ch in payload.get('channels', []) if ch.get('enabled')]
120         # Log des numéros de canaux activés pour validation
121         print(f"[WEBUI] Canaux activés: {[ch['channel'] for ch in enabled_channels]}")
122
123         # Création du chemin de fichier temporaire pour sauvegarde atomique
124         tmp_path = CONFIG_PATH + ".tmp"
125         # Ouverture du fichier temporaire en mode écriture UTF-8
126         with open(tmp_path, "w", encoding="utf-8") as f:
127             # Écriture des données JSON avec formatage lisible
128             json.dump(payload, f, ensure_ascii=False, indent=2)
129             # Force l'écriture vers le disque (buffer flush)
130             f.flush()
131             # Synchronisation système pour garantir l'écriture physique
132             os.fsync(f.fileno())
133         # Remplacement atomique du fichier de configuration
134         os.replace(tmp_path, CONFIG_PATH)
135
136         # Log de confirmation de sauvegarde réussie
137         print(f"[WEBUI] Configuration sauvegardée dans {CONFIG_PATH}")
138         # Retour de confirmation avec code HTTP 200 (succès)
139         return jsonify({"status": "ok"}), 200
140     except Exception as e:
141         # Log de l'erreur rencontrée lors de la sauvegarde
142         print(f"[WEBUI] Erreur sauvegarde config: {e}")
143         # Retour d'erreur avec code HTTP 400 (requête invalide)
144         return jsonify({"error": str(e)}), 400
145
146 # -----
147 # API REST pour monitoring de l'état système
148 # -----
149
150
151 @app.route("/api/last", methods=["GET"])
152 def get_last():
153     """
154     @brief Récupère le dernier état du système de mesure.
155     @details Lecture du fichier last_state.json contenant les dernières
156             mesures, températures et statuts. Gère le cas où le fichier
157             n'existe pas encore et retourne un état par défaut.
158
159     @return État JSON du système ou structure par défaut si fichier absent.
160     """
161     # Bloc de traitement avec gestion spécifique des exceptions
162     try:
163         # Ouverture du fichier d'état en mode lecture UTF-8
164         with open(STATE_PATH, "r", encoding="utf-8") as f:
165             # Chargement des données d'état depuis le fichier JSON
166             data = json.load(f)
167         # Log conditionnel seulement si données significatives présentes
168         if data.get('last_channel') is not None:
169             # Affichage du canal et température de la dernière mesure
170             print(f"[WEBUI] Données canal {data['last_channel']}: {data.get('last_temperature_c', 'N/A')}°C")
171         # Retour des données d'état avec code HTTP 200 (succès)

```

```

172         return jsonify(data), 200
173     except FileNotFoundError:
174         # Gestion spécifique du cas où le fichier d'état n'existe pas
175         print(f"[WEBUI] Fichier état non trouvé: {STATE_PATH}")
176         # Retour d'une structure d'état par défaut avec tous les champs
177         return jsonify({
178             "last_timestamp": None,
179             "last_channel": None,
180             "last_resistance_ohm": None,
181             "last_temperature_c": None,
182             "last_temperature_sim_c": None,
183             "last_error": "state file not found",
184             "channels": {}
185         }), 200
186     except Exception as e:
187         # Gestion de toute autre erreur de lecture d'état
188         print(f"[WEBUI] Erreur lecture état: {e}")
189         # Retour d'erreur avec code HTTP 500 (erreur serveur)
190         return jsonify({"error": str(e)}), 500
191
192
193 # -----
194 # API REST pour contrôle système
195 # -----
196
197 @app.route("/api/force-reload", methods=["POST"])
198 def force_reload():
199     """
200     @brief Force le rechargement de la configuration au prochain cycle.
201     @details Crée un fichier flag force_reload.flag avec horodatage
202             pour déclencher le rechargement de configuration par
203             le processus principal lors du prochain cycle de mesure.
204
205     @return Confirmation de demande ou message d'erreur avec code HTTP.
206     """
207     # Bloc de traitement avec gestion d'exceptions
208     try:
209         # Construction du chemin vers le fichier flag de rechargement
210         flag_path = os.path.join(BASE_DIR, "state", "force_reload.flag")
211         # Création du répertoire de destination si inexistant
212         os.makedirs(os.path.dirname(flag_path), exist_ok=True)
213         # Création du fichier flag avec horodatage ISO
214         with open(flag_path, "w") as f:
215             # Écriture de l'horodatage de la demande de rechargement
216             f.write(datetime.now().isoformat())
217
218         # Log de confirmation de demande de rechargement
219         print(f"[WEBUI] Rechargement config demandé")
220         # Retour de confirmation avec code HTTP 200 (succès)
221         return jsonify({"status": "reload requested"}), 200
222     except Exception as e:
223         # Log de l'erreur rencontrée lors de la création du flag
224         print(f"[WEBUI] Erreur demande rechargement: {e}")
225         # Retour d'erreur avec code HTTP 500 (erreur serveur)
226         return jsonify({"error": str(e)}), 500
227
228
229 # -----
230 # Point d'entrée principal du serveur
231 # -----
232
233 if __name__ == "__main__":
234     # Affichage des chemins de configuration pour débogage
235     print(f"BASE_DIR: {BASE_DIR}")
236     print(f"CONFIG_PATH: {CONFIG_PATH}")
237     print(f"STATE_PATH: {STATE_PATH}")
238     print(f"WEBUI_DIR: {WEBUI_DIR}")
239     # Démarrage du serveur Flask sur adresse IP fixe et port 8080
240     app.run(host="192.168.1.109", port=8080)

```